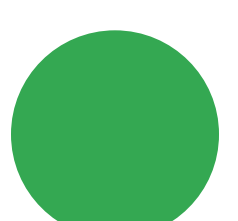




Core



concepts

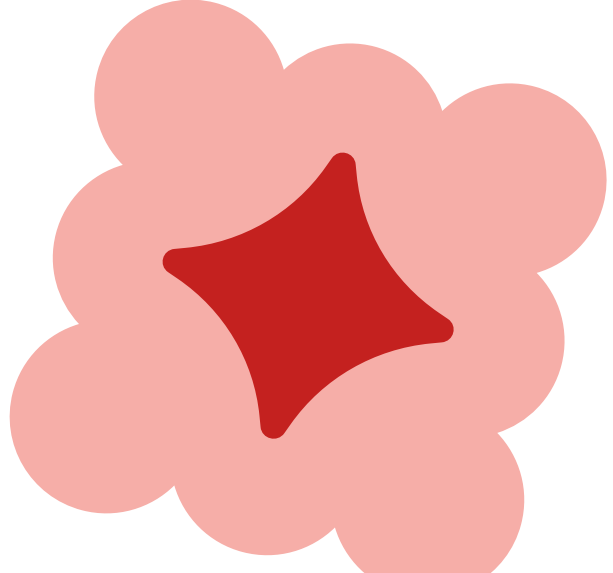


Introduction

You learned the three workflow agents:

```
Python
# Workflow agents
SequentialAgent # Run in order (A → B → C)
ParallelAgent   # Run together (A + B + C)
LoopAgent       # Repeat (A → B, repeat)
```

Now you will learn:

- How to decide which workflow agent fits your use case
 - How to coordinate state when using ParallelAgent
 - How to combine workflow agents effectively
- 
- 
- 
- 

Core concept: Choosing the right workflow

Decision guide

Situation	Use This	Why
Steps depend on each other	SequentialAgent	Each step needs previous output
Steps are independent	ParallelAgent	Faster - run at same time
Need to repeat until done	LoopAgent	Iterative refinement

Quick decision flowchart

None

Do steps depend on each other?

YES → SequentialAgent

NO → Can steps run at the same time?

YES → ParallelAgent

NO → Do you need to repeat?

YES → LoopAgent

NO → SequentialAgent (default)

Real-world examples

SequentialAgent examples:

- Customer intake → process request → send response
- Validate input → transform data → save to database
- Research topic → write article → edit article

ParallelAgent examples:

- Check multiple sources at once (news, academic, expert)
- Analyze document for sentiment and extract entities simultaneously
- Gather weather, traffic, and events data for trip planning

LoopAgent examples:

- Refine content until quality threshold met
- Retry API call until success
- Process items until queue empty

Core concept: State coordination in parallel

The problem with parallel agents

When parallel agents run at the same time, they can overwrite each other's state:

Python

```
# ❌ PROBLEM: Both agents write to the same key
agent_a = LlmAgent(output_key='result', ...) # Writes to 'result'
agent_b = LlmAgent(output_key='result', ...) # Also writes to 'result' - CONFLICT!

parallel = ParallelAgent(sub_agents=[agent_a, agent_b])
# One result is lost!
```

The solution: Distinct keys

Give each parallel agent a unique **output_key**:

Python

```
# ✅ CORRECT: Each agent has distinct key
agent_a = LlmAgent(output_key='result_a', ...) # Writes to 'result_a'
agent_b = LlmAgent(output_key='result_b', ...) # Writes to 'result_b'

parallel = ParallelAgent(sub_agents=[agent_a, agent_b])
# Both results preserved!
```



The aggregation pattern

After parallel agents complete, use a sequential agent to combine results:

Python

```
from google.adk.agents import LlmAgent, SequentialAgent, ParallelAgent

# Parallel research with distinct keys
research_a = LlmAgent(
    name='research_a',
    output_key='findings_a', # Distinct key
    instruction='Research benefits'
)

research_b = LlmAgent(
    name='research_b',
    output_key='findings_b', # Distinct key
    instruction='Research challenges'
)

parallel_research = ParallelAgent(
    name='parallel_research',
    sub_agents=[research_a, research_b]
)

# Aggregator combines parallel results
aggregator = LlmAgent(
    name='aggregator',
    instruction=''
    Combine findings:
    - Benefits: {findings_a}
    - Challenges: {findings_b}

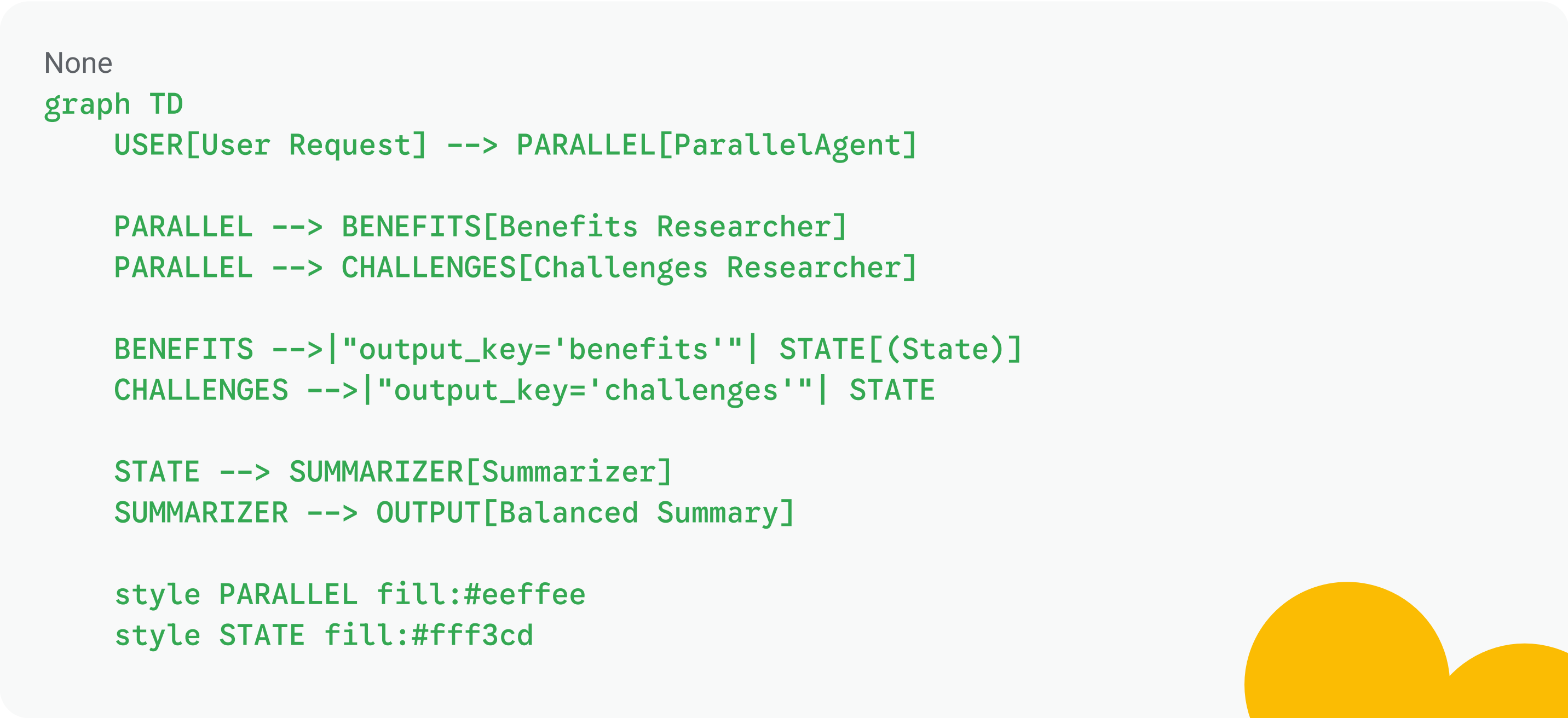
    Create balanced summary.
    ...
)

# Full workflow: parallel → aggregate
root_agent = SequentialAgent(
    name='research_workflow',
    sub_agents=[parallel_research, aggregator]
)
```


Key pattern:

- 1. Parallel agents with distinct `output_key` values
- 2. Followed by aggregator that reads all results via `{key}` templating
- 3. Wrapped in `SequentialAgent` to ensure proper ordering

The following diagram illustrates the aggregation pattern, showing how parallel agents write to distinct state keys that the summarizer combines:



Parallel research with proper state coordination: each agent writes to a distinct key, and the summarizer combines the results.

